الجامعة الافتراضية السورية
SYRIAN VIRTUAL UNIVERSITY

وظيفة مقرر IPG203

نظام ادارة الموظفين

Employee Management System

تطبيق متكامل لمبادئ البرمجة غرضية التوجه (OOP) بلغة C#

الأسم:	عامر بشار خراط
الصف:	C6
ID:	amer_203089

مدرس المادة:د.محمد معاذ الخباز

1 نظرة عامة على المشروع

هو تطبيق سطح مكتب مبني بلغة C# باستخدام Windows Forms، يهدف الى ادارة بيانات الموظفين باختلاف انواعهم، واحتساب الرواتب، وتسجيل البيعات، وتصدير التقارير الى Excel. يطبق المشروع بشكل كامل ومتكامل جميع مبادئ البرمجة غرضية التوجه (OOP).
نظام ادارة الموظفين (Employee Management System)

الوصف	الميزة
اضافة وتعديل وحذف الموظفين بثلاثة انواع	ادارة الموظفين
حساب الراتب الاساسي + المكافآت - الخصومات	احتساب الرواتب
خصم بالساعات او الايام مع حساب تلقائي	خصم الغياب
تسجيل بيعات الموظفين مع مكافاة اختيارية	تسجيل البيعات
تصدير كشف الرواتب وسجل البيعات بتنسيق RTL	تصدير Excel
حفظ وتحميل البيانات تلقائيا من ملف JSON	حفظ البيانات

2 هيكل المجلدات والملفات

تم تنظيم المشروع في ثلاثة مجلدات رئيسية وفق مبدأ Separation of Concerns:

مبدأ OOP	المحتوى	الملف
`Abstraction`	الواجهة الاساسية	`Models/IWorker.cs`
`Abstract Class`	الفئة المجردة الاساسية	`Models/Employee.cs`
`Inheritance`	موظف دوام كامل	`Models/FullTimeEmployee.cs`
`Inheritance`	موظف دوام جزئي	`Models/PartTimeEmployee.cs`
`Inheritance`	موظف متعاقد	`Models/Contractor.cs`
`Delegates & Events`	قسم + حدث التنبيه	`Models/Department.cs`
`Encapsulation`	سجل بيعة واحدة	`Models/SaleRecord.cs`
`Static Class`	التحقق من صحة البيانات	`Utilities/Validator.cs`
`Static Class`	حفظ وتحميل JSON	`Data/DataService.cs`
`Static Class`	تصدير Excel	`Data/ExcelExportService.cs`
`Polymorphism Usage`	نماذج واجهة المستخدم	`Forms/`

تطبيق مبادئ البرمجة غرضية التوجه 3

1. التجريد (Abstraction)

الواجهة وتحتوي على الخصائص المشتركة لجميع انواع الموظفين مع دالة CalculateSalary() المجردة التي يجب على كل فئة فرعية تعريفها.
تم تطبيق التجريد على مستويين: واجهة IWorker تعرف العمليات الاساسية المجردة، وفئة مجردة Employee تطبق هذه

الواجهة: IWorker

```
// Models/IWorker.cs
public interface IWorker
{
double CalculateSalary(); // حساب الراتب
string GetInfo(); // معلومات الموظف
void PerformTask(); // مهمة خاصة بنوع الموظف
}
```

الفئة المجردة: Employee

```
// Models/Employee.cs
public abstract class Employee : IWorker
{
private string _id;
private string _name;
private string _department;
// دالة مجردة – يجب تعريفها في كل فئة فرعية
public abstract double CalculateSalary();
// الراتب النهائي = الاساسي + المكافاة - الخصم
public double GetFinalSalary() => CalculateSalary() + CompanyBonus - Deduction;
}
```

2. الوراثة وتعدد الاشكال (Inheritance & Polymorphism)

نها تعيد تعريف CalculateSalary() بطريقة مختلفة. فئة Department تطبق تعدد الاشكال عند احتساب اجمالي الرواتب عبر قائمة الموظفين.
تم انشاء ثلاث فئات ترث من Employee، كل م

الفئة	نوع الموظف	حساب الراتب
`FullTimeEmployee`	دوام كامل	الاساسي + العلاوة
`PartTimeEmployee`	دوام جزئي	الساعات x سعر الساعة
`Contractor`	متعاقد	قيمة العقد الثابتة

تعريف CalculateSalary() في كل فئة:

```
// FullTimeEmployee.cs
public override double CalculateSalary() => _baseSalary + _bonus;
// PartTimeEmployee.cs
public override double CalculateSalary() => _hoursWorked * _hourlyRate;
// Contractor.cs
public override double CalculateSalary() => _contractAmount;
```

تعدد الاشكال في Department.GetTotalSalaries():

```
public double GetTotalSalaries()
{
double total = 0;
foreach (var emp in _employees) // قائمة من Employee (الفئة الاساسية)
{
double salary = emp.GetFinalSalary(); // POLYMORPHISM: يستدعي CalculateSalary() المناسبة
total += salary;
if (salary > SalaryThreshold)
OnHighSalaryDetected?.Invoke(emp.Name, salary); // EVENT
}
return total;
}
```

3. التغليف (Encapsulation)

ويتم الوصول اليها فقط عبر خصائص (Properties) تتحكم في صحة البيانات. الخاصية Id للقراءة فقط ولا يمكن تعديلها بعد الانشاء.
جميع حقول الفئات خاصة (private)،

```
// Employee.cs
private string _id; // حقل خاص
private string _name;
public string Id => _id; // قراءة فقط (read-only property)
public string Name
{
get => _name;
set {
if (!Validator.IsValidName(value))
throw new ArgumentException("الاسم يجب ان يكون بين 2 و50 حرفا.");
_name = value;
}
}
// تعديل المعرف عبر دالة محكومة فقط
public void SetId(string newId) {
if (newId.Length < 3) throw new ArgumentException("...");
_id = newId.Trim().ToUpper();
}
```

الخاصية	النوع	الوصول	التحقق
Id	string	قراءة فقط	مولد تلقائيا (GUID)
Name	string	قراءة/كتابة	2-50 حرف
HireDate	DateTime	قراءة فقط	لا يكون في المستقبل
BaseSalary	double	قراءة/كتابة	قيمة موجبة <= 999999
ContractEndDate	DateTime	قراءة فقط	يحسب من مدة العقد

4. التفويض والاحداث (Delegates & Events)

الرواتب اذا تجاوز راتب اي موظف الحد المحدد. تستخدم MainForm هذا الحدث لعرض تنبيه في سجل الاحداث (Delegates Events).
في فئة Department تم تعريف تفويض SalaryAlertHandler وحدث OnHighSalaryDetected. يطلق هذا الحدث تلقائيا عند احتساب

```
// Department.cs
// 1. تعريف التفويض //
public delegate void SalaryAlertHandler(string employeeName, double salary);
// 2. اعلان الحدث //
public event SalaryAlertHandler? OnHighSalaryDetected;
// 3. اطلاق الحدث عند تجاوز الحد //
if (salary > SalaryThreshold)
OnHighSalaryDetected?.Invoke(emp.Name, salary);
// MainForm.cs — الاشتراك في الحدث
_department.OnHighSalaryDetected += (name, sal) => {
```

```
LogEvent($"تحذير: راتب {name} مرتفع: {sal:F2}$");
};
```

الدور	الاسم	العنصر
يعرف توقيع الدالة المستدعاة	`SalaryAlertHandler`	التفويض
يطلق عند تجاوز حد الراتب	`OnHighSalaryDetected`	الحدث
يستقبل الحدث ويسجله في السجل	`MainForm`	المشترك

5. الفئات والاجراءات الساكنة (Static Classes & Members)

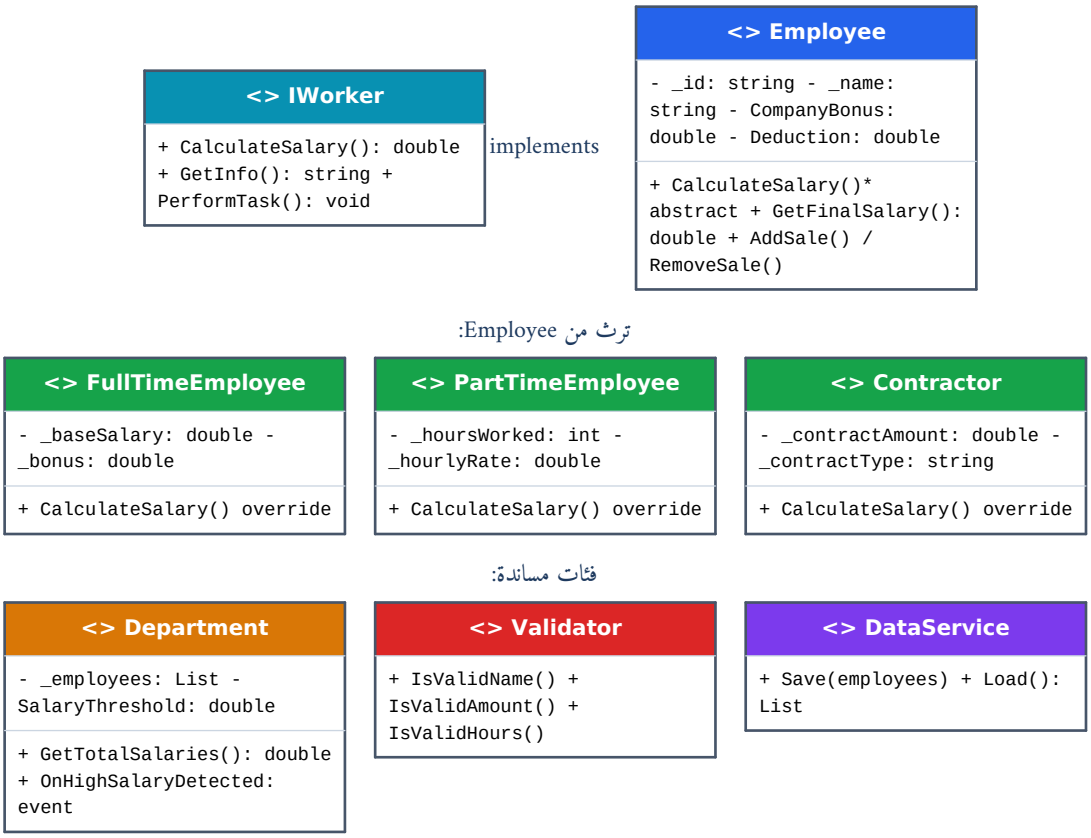
JSON، وExcelExportService لتصدير التقارير. كما تحتوي Employee على خاصية ساكنة TotalCreated تتتبع عدد الموظفين المنشاين.
يحتوي المشروع على ثلاث فئات ساكنة: Validator للتحقق من صحة البيانات، DataService لحفظ وتحميل البيانات من

```
// Utilities/Validator.cs — Static Class
public static class Validator
{
public static bool IsValidName(string? name)
=> !string.IsNullOrWhiteSpace(name)
&& name.Trim().Length >= 2 && name.Trim().Length <= 50;
public static bool IsValidAmount(double amount)
=> amount >= 0 && amount <= 999_999;
public static bool IsValidHours(int hours)
=> hours >= 1 && hours <= 744;
}
// Employee.cs — Static Property
private static int _totalCreated = 0;
public static int TotalCreated => _totalCreated;
```

الفئة	النوع	الوظيفة
`Validator`	`static class`	التحقق من صحة الاسم والقيم المالية والساعات
`DataService`	`static class`	حفظ وتحميل بيانات الموظفين من JSON
`ExcelExportService`	`static class`	تصدير كشف الرواتب وسجل البيعات الى Excel
`Employee.TotalCreated`	`static prop.`	عدد الموظفين المنشاين في جلسة التشغيل

4 مخطط UML المبسط

يوضح المخطط التالي العلاقات بين الفئات الرئيسية في المشروع:

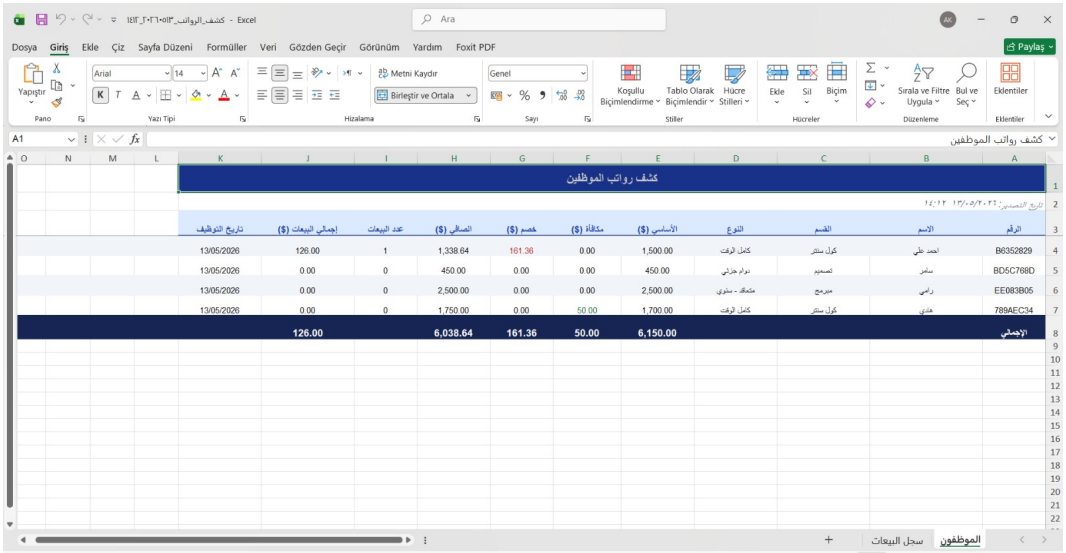


* الدالة المجردة يجب تعريفها في كل فئة فرعية.

لقطات الشاشة (Screenshots) 5

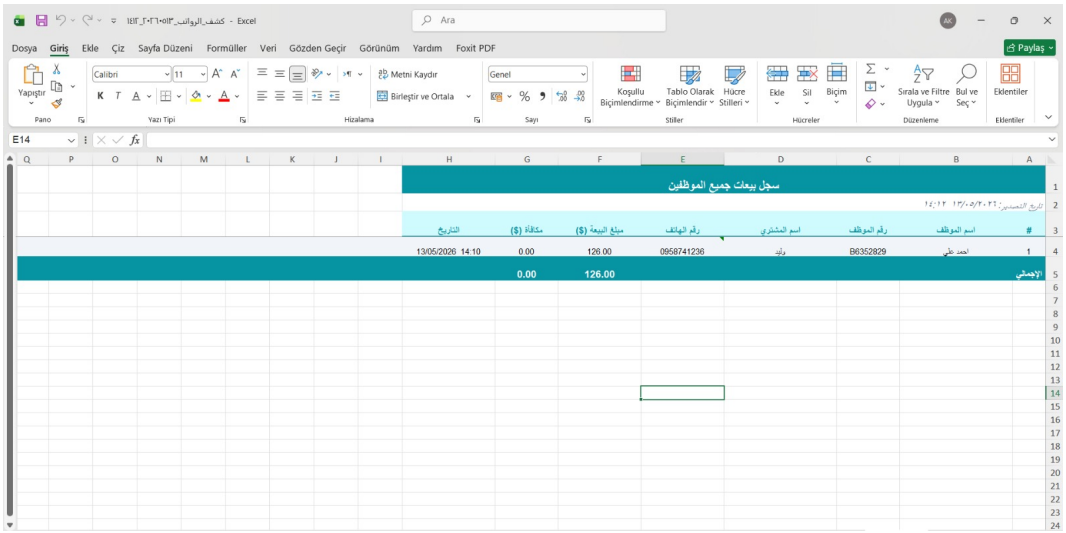
تظهر الصور التالية واجهة البرنامج وميزاته مع شرح لكل شاشة:

الشاشة 1: كشف الرواتب في Excel



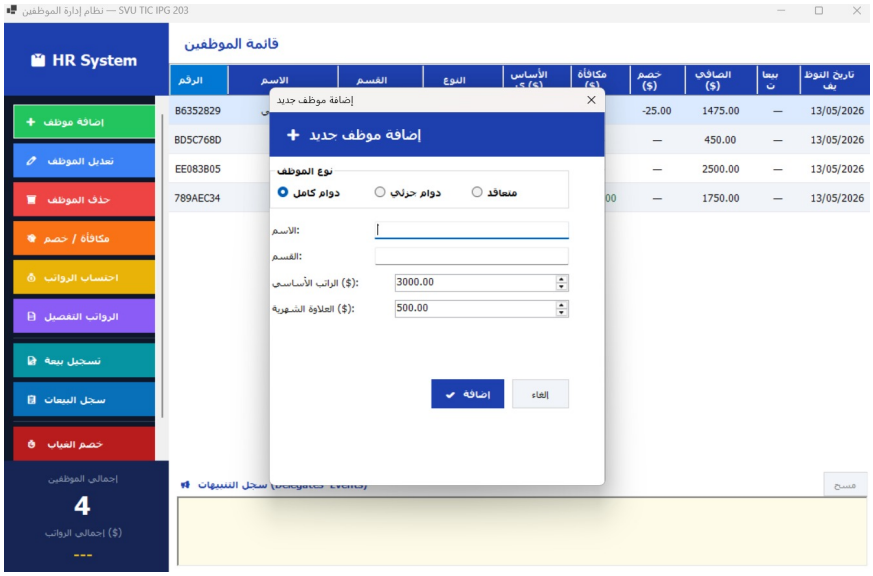
ورقة كشف رواتب الموظفين المصدرة الى Excel بتنسيق RTL والمجاميع التلقائية.

الشاشة 2: سجل البيعات في Excel



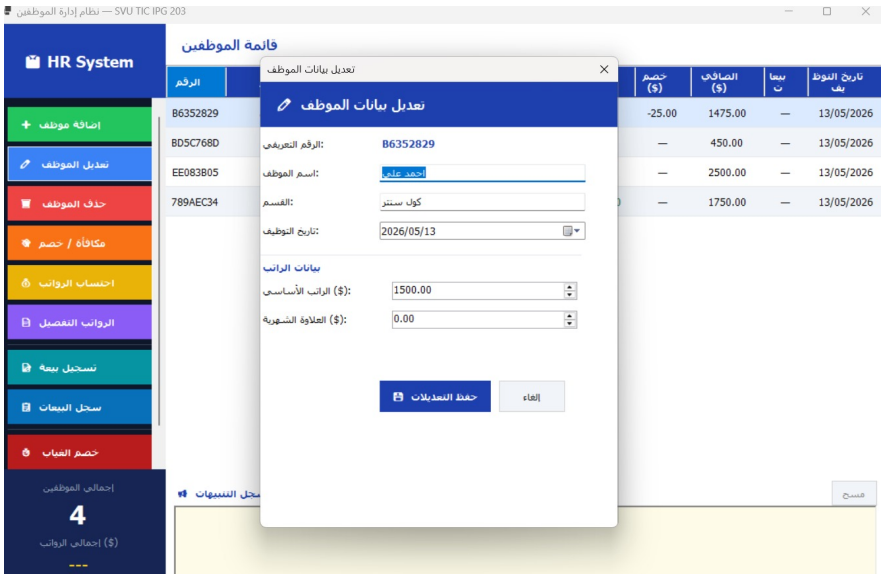
ورقة سجل بيعات جميع الموظفين المصدرة الى Excel.

الشاشة 3: اضافة موظف جديد



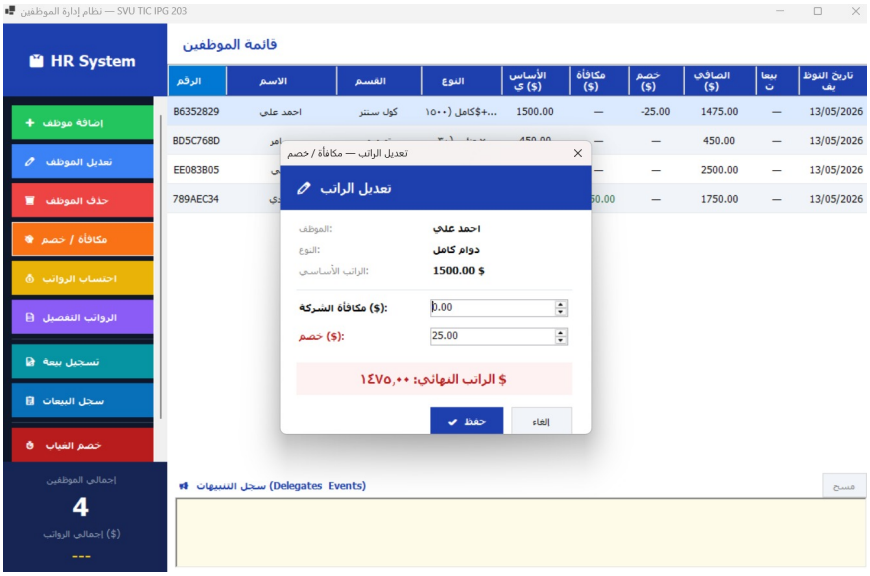
نافذة اضافة موظف جديد - تدعم ثلاثة انواع: دوام كامل، دوام جزئي، متعاقد.

الشاشة 4: تعديل بيانات الموظف



نافذة تعديل بيانات موظف موجود مع حقول الراتب الاساسي والعلاوة الشهرية.

الشاشة 5: تعديل الراتب - مكافاة / خصم



نافذة تعديل مكافاة الشركة والخصم مع عرض الراتب النهائي فوريا.

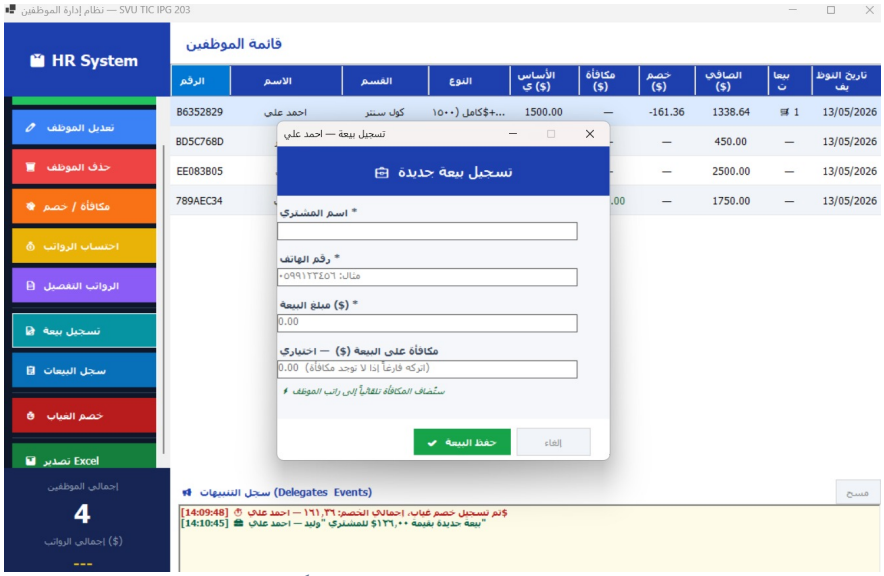
الشاشة 6: خصم الغياب بالساعات والايام

نافذة خصم الغياب: تحسب قيمة الساعة واليوم وتطبق الخصم على الراتب.

الشاشة 7: قائمة الموظفين - الشاشة الرئيسية

الشاشة الرئيسية تظهر جميع الموظفين مع الرواتب والخصومات والمكافآت.

الشاشة 8: تسجيل بيعة جديدة



نافذة تسجيل بيعة جديدة للموظف مع خيار اضافة مكافاة على البيعة.

الشاشة 9: سجل البيعات للموظف

نافذة سجل البيعات الخاصة بموظف محدد مع خيار التعديل والحذف.

رابط المشروع 6

رابط مشروع GitHub:

```
https://github.com/AmerKh0/SVU_IPG_203_amer_203089
```